Control Unit:
↳ Initiates a sequences of micro-operations

Types of CU:

- Hard-Wired
  ↳ The control logic is implemented using gates, F/Fs, decoders etc.

  \+ Fast ops
  − Wiring change (if design needs to be modified

- Micro-programmed
  ↳ The control information is stored in control memory, and control memory is programmed to initiate the required seq of micro-ops

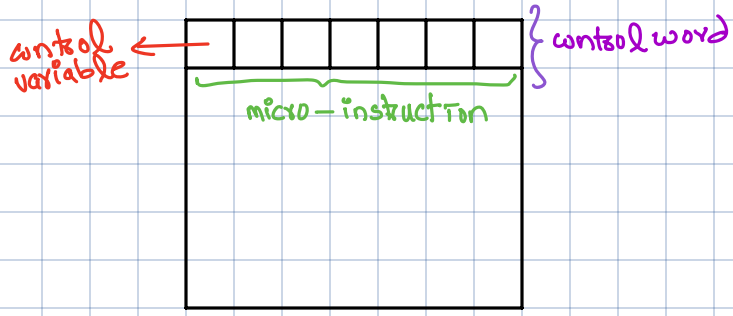
  \+ Any req change can be done by updating micro-program in control memory

  − Slow operation

Control Word:
↳ The control variables at any given time can be represented by a string of 1's & 0's.
↳ Each word in control memory contains a micro-instruction

Micro-instruction
↳ Specifies one or more micro-ops

Micro-program:
↳ A sequence of micro-instruction



Micro-programed CU:

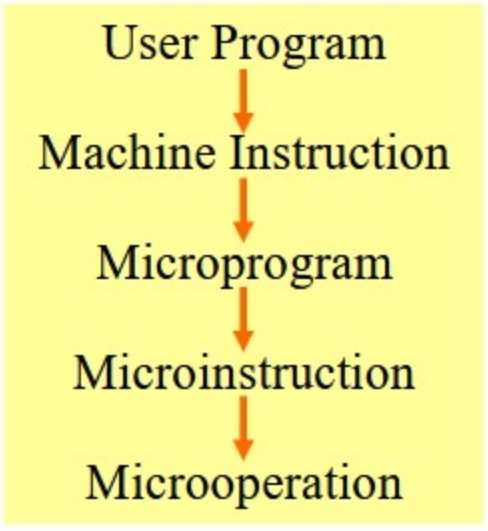
↳ A CU whose binary control variables are stored in control memory

- Static Micro-programming
  ↳ Since alterations of micro-program are not needed once CU is in operation. So it can be made permanent
  ↳ Control memory = ROM → Commonly used
    ↳ Fast & Cheaper
    ↳ Prevents accassional user from changing the architecture of the system

- Dynamic Micro-programming:
  ↳ Micro-program initially loaded from Aux memory like magnetic disk.
  ↳ Control memory = RAM

↳A computer that employs a micro-programmed control unit will have two separate memories:

1. Main Memory → Store user program (Machine instructions)
2. Control Memory
   ↳ To store micro-programs (micro-instructions)

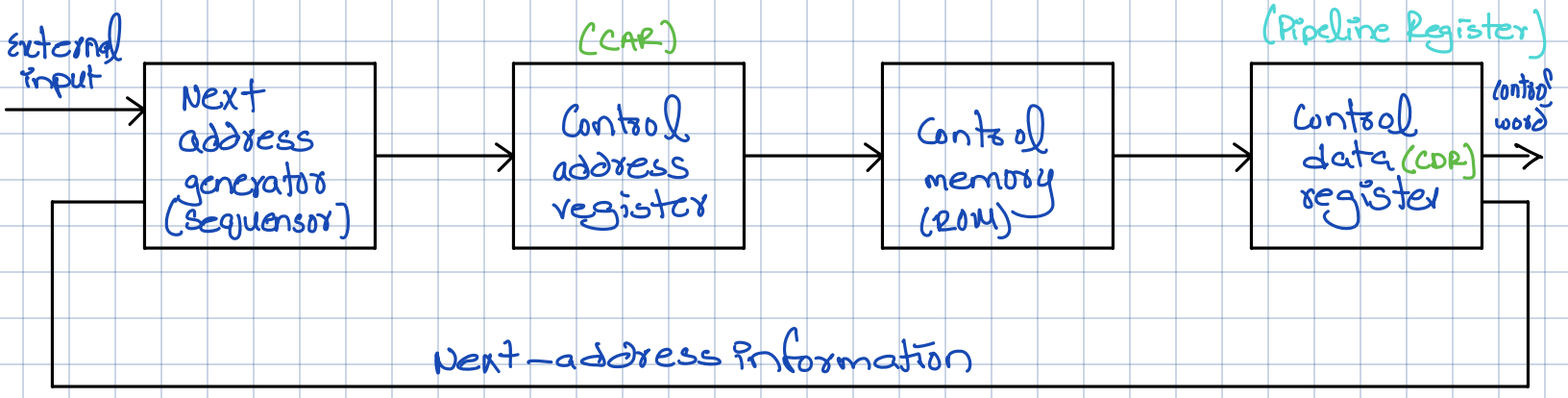


User writes Machine instruction

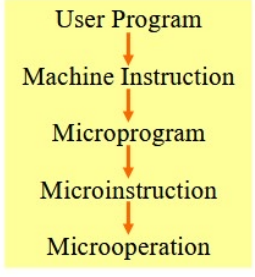
Machine instruction calls it's micro-program to execute the machine instruction

Micro-program has micro-instructions

Each micro-instruction has micro-operations.



- Microprogrammed control Organization : ***Fig. 7-1***
  - 1) Control Memory
    - » A memory is part of a control unit : *Microprogram*
    - » Computer Memory (*employs a microprogrammed control unit*)
      - Main Memory : for storing user program (*Machine instruction/data*)
      - Control Memory : for storing microprogram (*Microinstruction*)
  - 2) Control Address Register
    - » Specify the address of the microinstruction
  - 3) Sequencer (= *Next Address Generator*)
    - » Determine the address sequence that is read from control memory
    - » Next address of the next microinstruction can be specified several way depending on the sequencer input : *p. 217, [1, 2, 3, and 4]*



- 4) Control Data Register (= *Pipeline Register*)
  - » Hold the microinstruction read from control memory
  - » Allows the execution of the microoperations specified by the control word ***simultaneously*** with the generation of the next microinstruction

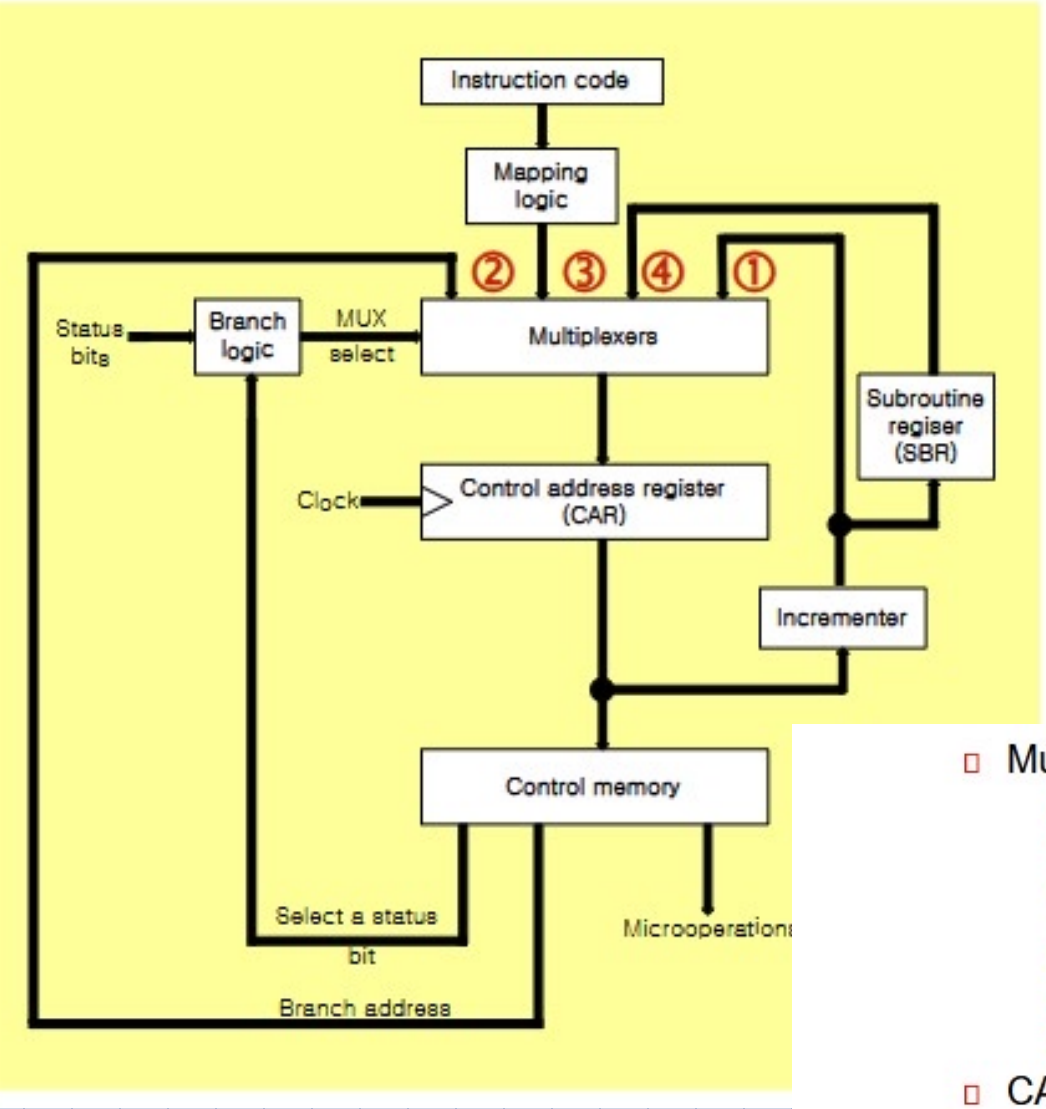
Routine

↳ Micro-instructions are stored in Micro-program in groups called Routine.

↳ Each computer instructions has it's own micro-program routine in control memory to generate the micro-ops that exe the instruction.

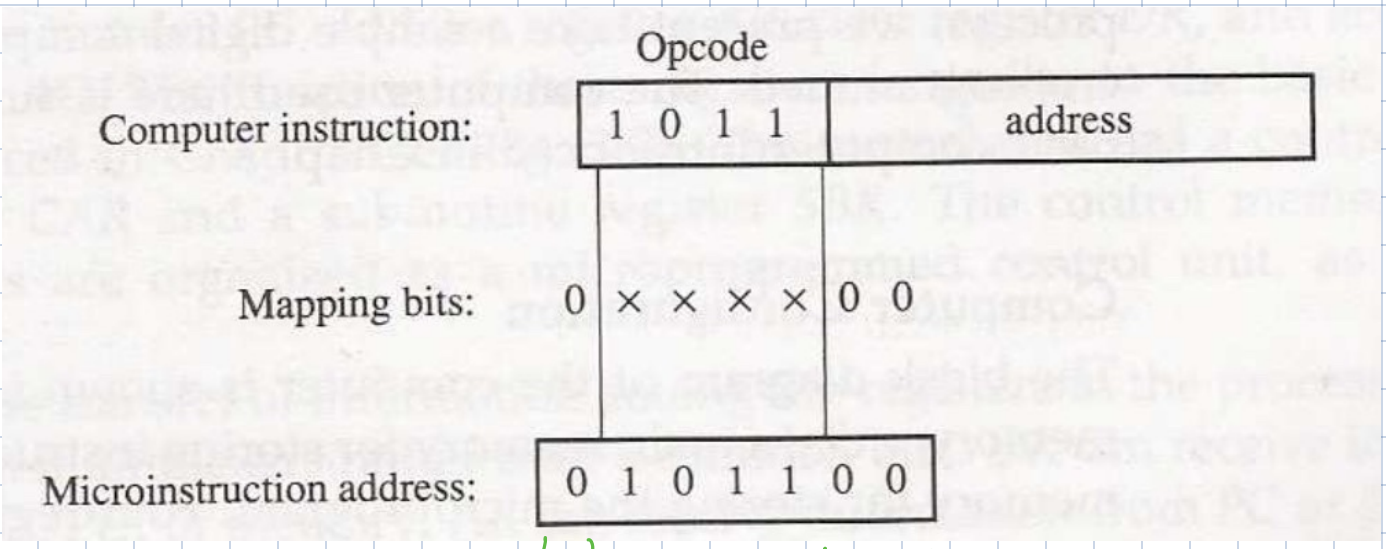
Mapping:

↳ Instruction Code ⟶ Address in Control memory (Where micro-program is stored)



- Multiplexer
  - ① CAR Increment
  - ② JMP/CALL
  - ③ Mapping
  - ④ Subroutine Return
- CAR : Control Address Register
  - » CAR receive the address from 4 different paths
    1) Incrementer
    2) Branch address from control memory
    3) Mapping Logic
    4) SBR : Subroutine Register
- SBR : Subroutine Register
  - » Return Address can not be stored in ROM
  - » Return Address for a subroutine is stored in SBR

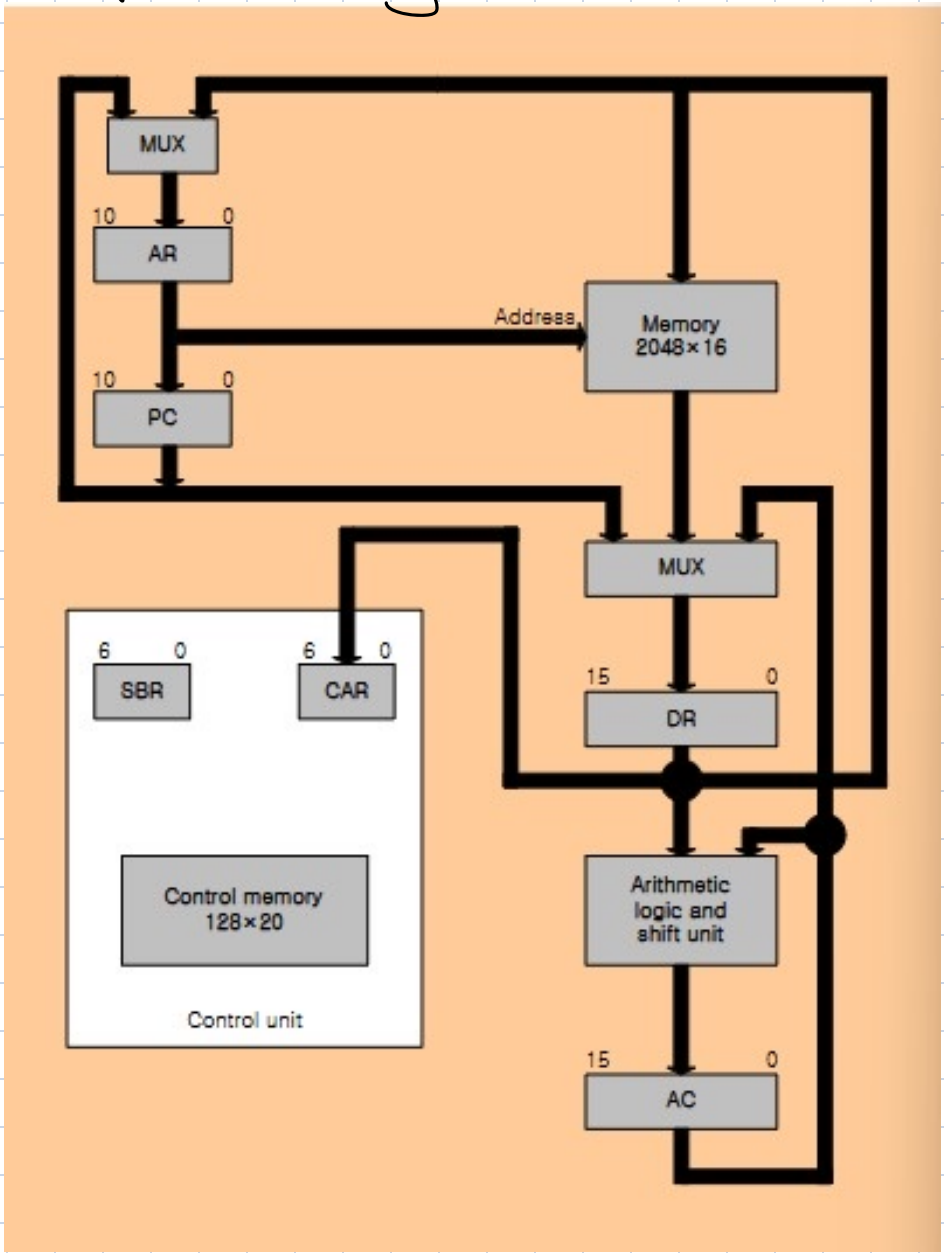
Mapping:



↳ Implemented by ROM or PLA

Total 6 potential micro-instructions

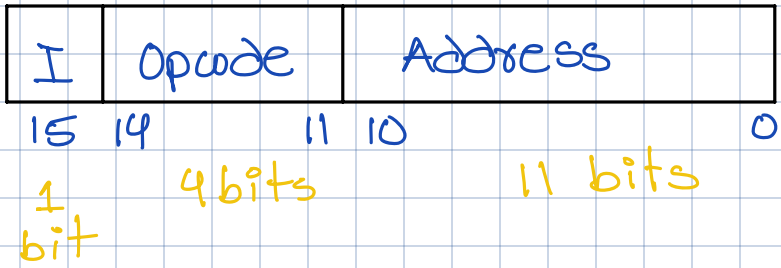
Computer configuration



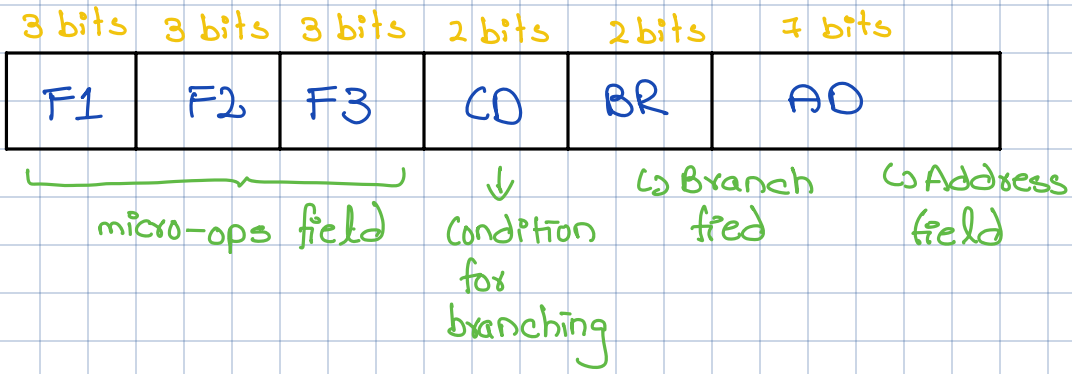
↳ Consists of

1. 2 memories
   - ↳ Main memory
   - ↳ Control memory
2. 4 CPU Registers & ALU
3. 2 CU register
   - ↳ SBR
   - ↳ CAR

Machine instruction format



Micro-instruction format:



F1	Microoperation	Symbol
000	None	NOP
001	AC ← AC + DR	ADD
010	AC ← 0	CLRAC
011	AC ← AC + 1	INCAC
100	AC ← DR	DRTAC
101	AR ← DR(0-10)	DRTAR
110	AR ← PC	PCTAR
111	M[AR] ← DR	WRITE

F2	Microoperation	Symbol
000	None	NOP
001	AC ← AC - DR	SUB
010	AC ← AC ∨ DR	OR
011	AC ← AC ∧ DR	AND
100	DR ← M[AR]	READ
101	DR ← AC	ACTDR
110	DR ← DR + 1	INCDR
111	DR(0-10) ← PC	PCTDR

F3	Microoperation	Symbol
000	None	NOP
001	AC ← AC ⊕ DR	XOR
010	AC ← AC'	COM
011	AC ← shl AC	SHL
100	AC ← shr AC	SHR
101	PC ← PC + 1	INCPC
110	PC ← AR	ARTPC
111	Reserved	

- 2 bit Condition Fields : CD
  - » 00 : Unconditional branch, **U** = Always 1
  - » 01 : Indirect address bit, **I** = DR(15)
  - » 10 : Sign bit of AC, **S** = AC(15)
  - » 11 : Zero value in AC, **Z** = AC = 0
- 2 bit Branch Fields : BR
  - » 00 : **JMP**
    - Condition = 0 : 1 $CAR \leftarrow CAR + 1$
    - Condition = 1 : 2 $CAR \leftarrow AD$
  - » 01 : **CALL**
    - Condition = 0 : 1 $CAR \leftarrow CAR + 1$
    - Condition = 1 : 2 $CAR \leftarrow AD, SBR \leftarrow CAR + 1$
  - » 10 : **RET** 3 $CAR \leftarrow SBR$
  - » 11 : **MAP** 4 $CAR(2-5) \leftarrow DR(11-14), CAR(0,1,6) \leftarrow 0$

Save Return Address

Restore Return Address

- 7 bit Address Fields : AD
  - » 128 word : 128 X 20 bit

Symbolic Microinstruction

① Label Field : Terminated with a colon (:)

② Microoperation Field : one, two, or three symbols, separated by commas

③ CD Field : U, I, S, or Z

④ BR Field : JMP, CALL, RET, or MAP

① Label	② Microoperation	③ CD	④ BR	⑤ AD
	ORG 64			
FETCH:	PCTAR	U	JMP	NEXT
	READ, INCPC	U	JMP	NEXT
	DRTAR	U	MAP	0

⑤ AD Field

a. Symbolic Address : Label (= Address)

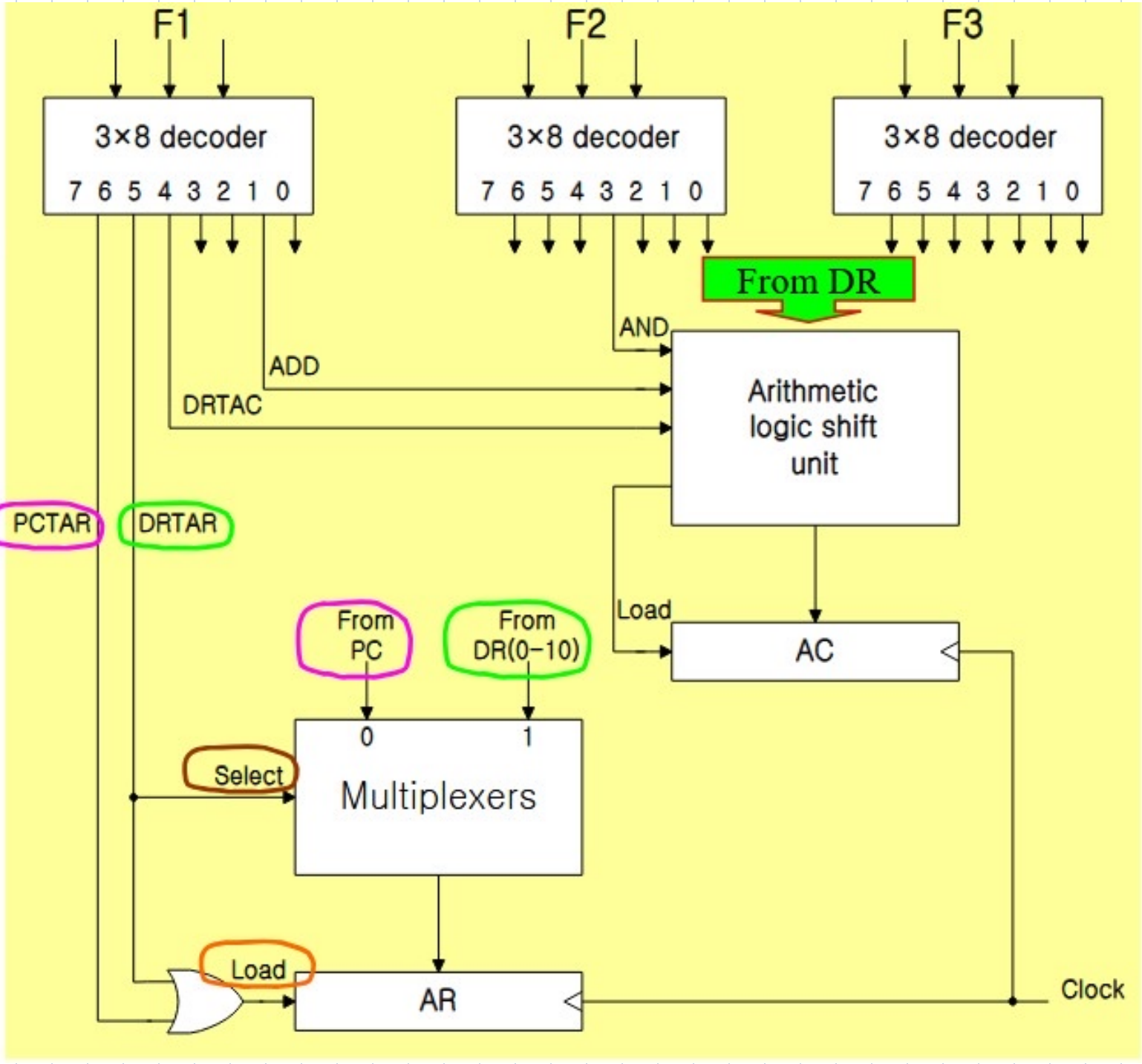
b. Symbol “NEXT” : next address

c. Symbol “RET” or “MAP” : AD field = 0000000

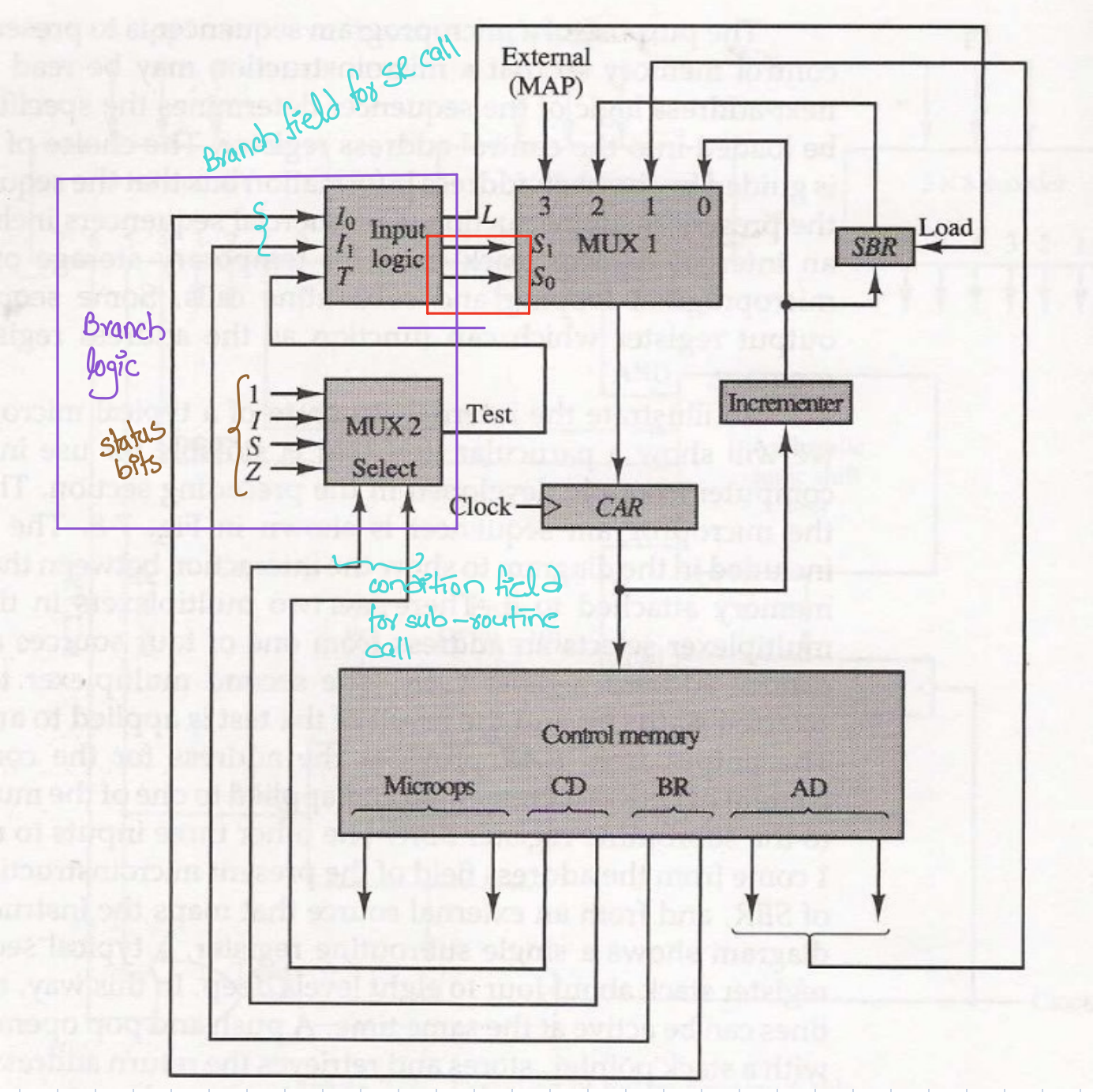
- ORG : Pseudoinstruction(*define the origin, or first address of routine*)

Binary address	F1	F2	F3	CD	BR	AD
1000000	110	000	000	00	00	1000001
1000001	000	100	101	00	00	1000010
1000010	101	000	000	00	11	0000000

Design of Control Unit:



Micro-program Sequencer:



Next micro-instruction logic

S_1S_0	Address Source
00	CAR + 1, In-Line
01	SBR RETURN
10	CS(AD), Branch or CALL
11	MAP

Condition & Branch control

I_1I_0T	Meaning	Source of Address	S_1S_0	L
000	In-Line	CAR+1	00	0
001	JMP	CS(AD)	01	0
010	In-Line	CAR+1	00	0
011	CALL	CS(AD) and SBR <- CAR+1	01	1
10x	RET	SBR	10	0
11x	MAP	DR(11-14)	11	0

Book Qs:

Q1.

↳ Micro-processor is a hard-ware component.

↳ Micro-program is a sequence of micro-instructions.

↳ Yes, we can design a CU without micro-program. Hardwired CU requires no micro-program.

↳ No, micro-programmed computers donot have to be a micro-processor

Q2.

- Hard-Wired
  - ↳ The control logic is implemented using gates, F/Fs, decoders etc.
  - \+ Fast ops
  - − Wiring change (if design needs to be modified

- Micro-programmed
  - ↳ The control information is stored in memory, and control memory is programmed to initiate the required seq of micro-ops
  - \+ Any req change can be done by updating micro-program in control memory
  - − Slow operation

↳ Hard-wired CU donot have control memory.

Q3.

Micro-operation: Elementary digital Computer op done in one clk pulse

Micro-instruction: Sequence of micro-operations

Micro-program/Micro-code: Sequence of micro-instructions

Q5.

32 total bits

16 bits for micro-operations

So, 32 − 16 = 16

As, $2^{10} = 1024$ words

16 − 10 = 6

(a) Branch address field: 10

Select field: 6

(b) $2^4 = 16$

So, 4 bits

(c) 6 − 4 = 2

Q4.

freq of each Clk:

$$\frac{1}{100 \times 10^{-9}} = 10\,MHz$$

Sequencer 40+10+40+10 ROM

CAR Data register

Q6.

(a) $2^{12} = 4096$

So, CAR has 12 bits

If OR is removed, we can use single phase clk

freq: $\frac{1}{90 \times 10^{-9}} = 11.1\,MHz$

40+10+40

(b) Each will be 12 bits as they are all addresses of control memory

(c) n kx1 Mux

12 4x1 Mux

Q7.

(a) 0010 (b) 1011 (c) 1111

00010000 01011100 01111100

Q8.

op-code: 6 bits

Total words: 2048

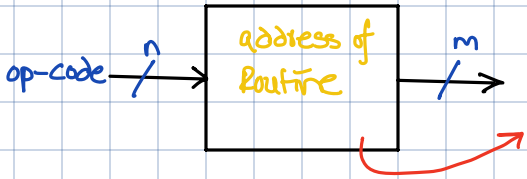
So, $2^{11} = 2048$

00 X X X X X X 000

Remaining bits

8 consecutive micro-instructions

Q9. ROM can be programmed to provide any desired address for a given inputs from the instruction



Contains pre-programmed addr of routine at each address

Q10.

↳ We need two MUX because we need branching capabilities & normal incremented address. Also, OR can have branch address or simply an operand.

↳ Other options instead of MUX include tri-state buffers, or gate logic.

Q11.

(a) $AC \leftarrow AC + 1$, $DR \leftarrow DR + 1$

011 110 000

(b) $DR \leftarrow M[AR]$, $PC \leftarrow PC + 1$

000 100 101

(c) $AC \leftarrow DR$ $DR \leftarrow AC$

100 101 000

Q12.

(a) $DR \leftarrow M[AR]$, $PC \leftarrow PC + 1$ 000 100 101

(b) $DR \leftarrow AC$, $AC \leftarrow DR$ 100 101 000

(c) $PC \leftarrow AR$, $AC \leftarrow DR$, $M[AR] \leftarrow DR$ 100 110

111 } error can't have two F1

Q13.

	Read	I	Call	INDR2
ADD:	ADD	U	Jmp	FETCH
INDR2:	DRTAR	U	Jmp	NEXT
	READ	U	Ret	

{ Read from memory in DR & go to subroutine INDR2 if I is 1

Q14.

```
ORG 40
NOP         S   JMP     FETCH
NOP         Z   JMP     FETCH
NOP         I   CALL    INDRCT
ARTPC       U   JMP     FETCH
```

(a) if we have a signed or zero value we skip the current instruct. else we read address in AR if I is 1 & then we go to next instruction.

(b)

40:	000	000	000	10	00	64
41:	000	000	000	11	00	64
42:	000	000	000	01	01	67
43:	000	000	110	00	00	64

Q15.

Address	F1	F2	F3	Binary Microprogram
60	010	000	010	00 00 10000 11
61	111	100	000	01 01 10000 00
62	001	001	000	10 10 01111 11
63	101	110	000	11 11 01100 00

(a) ORG 60

(b)

CLRAC, COM → Accessing AC at the same time — U

Write, Read → Accessing DR at same time — I

ADD, SUB → Accessing AC at the same time — S

DRTAR, INCOR → Accessing DR at same time — Z

Unconditional jump to INDRCT routine with no way to return

Jmp 67

Call → Behaves as jmp when I=0 — 64

RET → returning to whatever's in SBR independent of S — NEXT

MAP 60 → executed independent of Z & 60.

Q16.

```
      ORG 16
AND:  NOP     I   Call  INDRCT
      READ    U   Jmp   NEXT
      AND     U   Jmp   FETCH
```

```
        ORG 20
SUB:    NOP         I   Call   INDRCT
        READ        U   Jmp    NEXT
        SUB         U   Jmp    FETCH

        ORG 24
ADM:    NOP         I   Call   INDRCT
        READ        U   Jmp    NEXT
    DRTAC, ACTDR    U   Jmp    NEXT
        ADD         U   Jmp    NEXT
    DRTAC, ACTDR    U   Jmp    NEXT
        WRITE       U   Jmp    FETCH

        ORG 28
BTCL:   NOP         I   Call   INDRCT
        READ        U   Jmp    NEXT
    DRTAC, ACTDR    U   Jmp    NEXT
        COM         U   Jmp    NEXT
        AND         U   Jmp    FETCH

        ORG 32
BZ:     NOP         Z   Jmp    BRC
        NOP         U   Jmp    FETCH

BRC:    NOP         I   Call   INDRCT
        ARTPC       U   Jmp    FETCH

        ORG 36
SEQ:    NOP         I   Call   INDRCT
        READ        U   Jmp    NEXT
    DRTAC, ACTDR    U   Jmp    NEXT
        XOR         U   Jmp    NEXT
    DRTAC, ACTDR    Z   Jmp    EQUAL
    NOP             U   Jmp    FETCH

EQUAL:  INCPC       U   Jmp    FETCH

        ORG 40

BPNZ:   NOP         S   Jmp    FETCH
        NOP         Z   Jmp    FETCH
```

AC > 0

NOP	I	Call	INDRCT → AR=[AR]
ARTPC	U	Jmp	FETCH

Q17.

ISZ: NOP	I	Call	INDRCT
READ	U	Jmp	NEXT
INCDR	U	Jmp	NEXT
DRTAC, ACTDR	U	Jmp	NEXT
DRTAC, ACTDR	Z	Jmp	SKIP
WRITE	U	Jmp	FETCH

DR ← M[AR]
DR ← DR+1
if(DR=0) PC ← PC+1,
M[AR] ← DR

SKIP: INCPC, WRITE	U	Jmp	FETCH

Q18.

BSA: NOP	I	Call	INDRCT
PCTDR, ARTPC	U	Jmp	NEXT
WRITE, INCPC	U	Jmp	FETCH

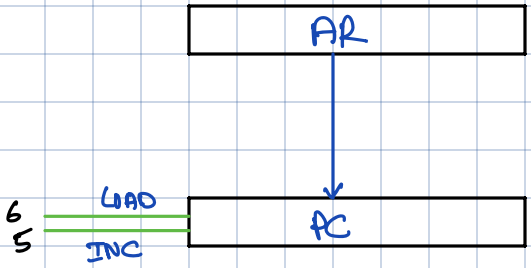
M[AR] ← PC
AR ← AR+1
PC ← AR

Q19.

From table,

5 : PC ← PC+1

6 : PC ← AR



Q20.

↳ 1 bit from each is for None

Q21. 16 registers need 4 bits

ALU needs 5 bits

Shifter needs 3 bits

	4	4	4	5	3
(a),(b)	SRC1	SRC2	DEST	ALU	SHIFT
(c)	0101	0110	0100	00100	000

Q22.

BR field

		I_2	I_1	I_0	T	S_1	S_0	L	
0	1	0	0	0	0	0	1	0	Jmp to AD
0	0	0	0	0	1	0	0	0	CAR ← CAR + 1
0	1	0	0	1	0	0	1	0	Jmp to AD (As I_2 is X)
0	1	0	0	1	1	0	1	0	Jmp to AD (T is X)
0	0	0	1	0	0	0	0	0	CAR ← CAR+1
0	1	0	1	0	1	0	1	0	Jmp to AD
1	0	0	1	1	0	1	0	0	RET
1	0	0	1	1	1	1	0	0	RET
0	0	1	0	0	0	0	0	0	CAR ← CAR+1
0	0	1	0	0	1	0	0	0	CAR ← CAR+1
0	1	1	0	1	0	0	1	0	Jmp to AD
0	1	1	0	1	1	0	1	0	Jmp to AD
0	0	1	1	0	0	0	0	0	CAR ← CAR+1
0	1	1	1	0	1	0	1	1	
1	1	1	1	1	0	1	1	0	
1	1	1	1	1	1	1	1	0	

S_1

I_2I_1 \ I_0T	00	01	11	10
00	0	0	0	0
01	0	0	1	1
11	0	0	1	1
10	0	0	0	0

$S_1 = I_1 I_0$

S_0

I_2I_1 \ I_0T	00	01	11	10
00	1	0	1	1
01	0	1	0	0
11	0	1	1	1
10	0	0	1	1

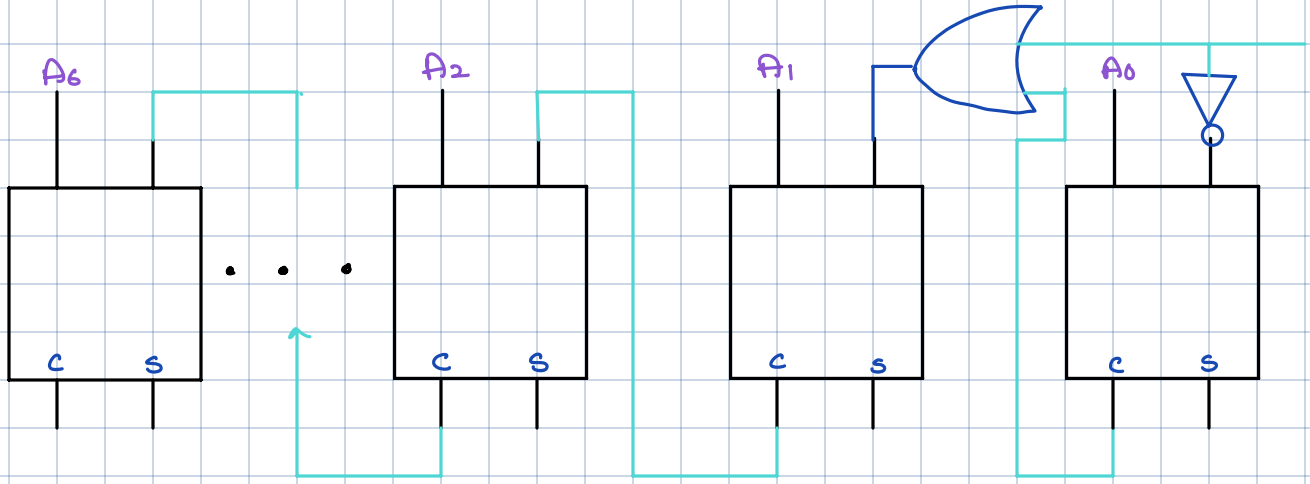
$S_0 = I_2 I_0 + \bar{I}_1 I_0 + I_1 \bar{I}_0 T + \bar{I}_2 \bar{I}_1 \bar{T}$

L

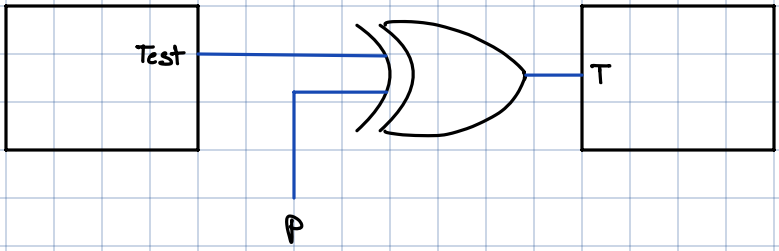
I_2I_1 \ I_0T	00	01	11	10
00				
01				
11		1		
10				

$L = I_2 I_1 \bar{I}_0 \bar{T}$

Q23.



Q24.



Test ⊕ 0 = Test

Test ⊕ 1 = $\overline{\text{Test}}$

P is used to determine polarity of selected status bit